Contents

Next-Generation Data Race Detection 1
About the Project 1
Project 1: L0-Filter: Compiler-Inlined Thread-Local Memoization 1
Project 2: Ownership Meta-Shadow Memory (Adaptive State Instrumentation) 2
Project 3. Extending ThreadSanitizer (TSan) to Detect the ABA Problem in Non-blocking Algorithms 3

Next-Generation Data Race Detection

About the Project

Multithreaded C/C++ programs are highly susceptible to data races—one of the most insidious and dangerous classes of concurrency bugs. The industry standard for detecting these bugs is **ThreadSanitizer (TSan)**, a dynamic analysis tool integrated into LLVM/Clang and GCC. However, TSan introduces a massive 5× to 15× runtime overhead, preventing its use in production environments or heavy CI/CD pipelines (e.g., for Chromium or high-throughput databases).

Our team is developing cutting-edge compiler optimizations within the LLVM infrastructure to drastically reduce TSan's overhead. We are looking for motivated students to implement advanced research ideas. Successful projects will be submitted to top-tier international conferences (ECOOP, PLDI, ASPLOS) and potentially upstreamed into the mainline LLVM repository.

Project 1: L0-Filter: Compiler-Inlined Thread-Local Memoization

Context:
To detect races, TSan instruments almost every memory access. However, over 90% of these accesses are dynamically redundant: a thread often reads the same variable repeatedly in a loop or during graph traversal. Static analysis cannot eliminate these checks if the loop contains opaque function calls (due to the risk of hidden synchronization).

The Task:

Develop an ultra-fast, compiler-inlined software cache (N-way direct-mapped) using Thread-Local Storage (TLS).

- Write an LLVM Pass that replaces the unconditional TSan runtime call with 4–5 inlined ALU instructions (address hashing and cache checking).
- Modify the TSan C++ runtime to aggressively flush (zero out) this cache upon *any* synchronization event (mutexes, atomics) to guarantee soundness.
- Benchmark the Hit Rate and execution speedup on real-world projects (Redis, SQLite, FFmpeg).

Recommended Skills:

- Good knowledge of C++ and computer architecture (Cache lines, Branch Prediction).

- Basic understanding of LLVM IR and the LLVM Pass Framework.

Literature & Useful Links:

- Base paper: Compiling Away the Overhead of Race Detection
- *Paper:* What's the Optimal Performance of Precise Dynamic Race Detection? (ReX) — Explains dynamic redundancy.
- *LLVM Docs:* Writing an LLVM Pass & Thread-Local Storage.

Project 2: Ownership Meta-Shadow Memory (Adaptive State Instrumentation)

(Target: Strong Master / PhD Intern)

Context:
TSan stores the access history of the application in a highly packed "Shadow Memory" (mapping 8 application bytes to 32 shadow bytes). Modifying this core structure to add new states is practically impossible without breaking SIMD optimizations. However, a vast amount of heap memory is effectively "Owned" by a single thread (Thread-Private) or strictly Read-Only (Single-Writer/Multiple-Reader). If we could track this *per-object* at runtime, we could skip the expensive TSan core algorithm entirely for these safe objects.

The Task:
Design and implement a "Fast Filter" using a parallel **Meta-Shadow Memory** mapping.

- Implement a secondary shadow memory region in the TSan OS-level runtime (e.g., 1 byte of metadata per 8 bytes of app memory).
- Hook into the memory allocator interceptors (malloc/free) to tag newly allocated memory with the current Thread ID (Ownership) and State (Exclusive / Shared).
- Modify the LLVM Pass to inline a check against this Meta-Shadow memory: if the memory is currently marked "Exclusive" and the TID matches, skip the TSan call.
- Implement the "Revocation" logic: if a different thread accesses the memory, safely transition the Meta-Shadow state to "Shared" and fallback to standard TSan tracking.

Recommended Skills:

- Advanced C/C++ and low-level system programming.
- Knowledge of OS Memory Management (virtual memory, mmap, page faults).
- Ability to read and modify complex runtime systems (Compiler-RT).

Literature & Useful Links:

- Base paper: Compiling Away the Overhead of Race Detection
- *Paper:* ThreadSanitizer – data race detection in practice (Original TSan v2 algorithm).
- *Codebase:* Study compiler-rt/lib/tsan/rtl/tsan_platform.h (specifically the MemToMeta mapping logic) and tsan_shadow.h.

Project 3. Extending ThreadSanitizer (TSan) to Detect the ABA Problem in Non-blocking Algorithms

Domain & Keywords: Compilers, Concurrency, Dynamic Analysis, Non-blocking synchronization (lock-free/wait-free/obstruction-free), ABA
Complexity: Advanced
Target Components: LLVM (Core Library, Instrumentation), TSan RTL

1. Project Background

ThreadSanitizer (TSan) is the industry standard for dynamic data race detection in applications written in LLVM-compiled languages. It relies on compile-time instrumentation and a complex runtime library to track happens-before relationships using vector clocks.

However, currently, TSan focuses primarily on **data races** (unsynchronized access) and **deadlocks**. It does not natively detect the **ABA problem**, a classic anomaly in lock-free programming.

The ABA Problem:

Occurs when a thread reads a value `A` from a shared location, performs computations, and attempts to update the variable using `CAS` (Compare-And-Swap). Meanwhile, another thread changes the value from `A` to `B` and back to `A`. The `CAS` succeeds (since the value appears to be A), but the internal state of the application (e.g., memory pointers, stack depth) has changed, leading to silent data corruption or segmentation faults.

2. Problem Statement

Standard TSan detects memory access without locks. However, the ABA problem is technically "synchronized" via atomics, so TSan perceives the execution flow as valid, even if the algorithmic logic is flawed.

Currently, there is no lightweight, automated way in the LLVM ecosystem to flag potential ABA violations during testing without manually implementing version counters (tagging) in user code.

3. Project Goals

The goal is to implement an experimental feature in ThreadSanitizer to detect ABA scenarios at runtime.

The student is expected to:

1. **Research:** Analyze how TSan instruments multithreaded code and utilizes shadow memory to store metadata (thread IDs, timestamps).
2. **Design:** Propose a mechanism to track "versioning" or "modification counts" for atomic variables within TSan's shadow memory.
3. **Implementation:** Modify the LLVM compiler pass and TSan runtime to:
    - Intercept atomic loads and `cmpxchg` (`CAS`) operations.
    - Maintain a shadow "generation counter" for atomic memory locations.
    - Issue a warning if a CAS succeeds on value A, but the shadow counter indicates the variable was modified in the interim.
4. **Validation:** Create a suite of unit tests containing classic lock-free data structures (e.g. Treiber Stack, Michael-Scott Queue) with intentional ABA bugs and verify their detection.

4. Technical Approach (Starting Point)

- **Shadow Memory Extension:** TSan maps application memory to shadow memory. The student needs to repurpose unused bits in the shadow cell or allocate extra metadata to store a monotonically increasing modification_id for atomics.
- **Instrumentation:**
    - On `atomic_store / atomic_exchange`: Increment the shadow modification_id.
    - On `atomic_load`: The runtime must return not just the value, but also record the modification_id active at the moment of the read (stored in thread-local storage).
    - On `atomic_compare_exchange`: Verify if the current global `modification_id` matches the one "seen" during the corresponding `atomic_load`.
- *Note:* This is a heuristic approach. Since load and CAS are distinct instructions, the primary algorithmic challenge will be associating them correctly within the analyzer.

5. Prerequisites

- **Languages:** Strong proficiency in C++ (C++14/17), C.
- **Knowledge:**
    - Deep understanding of Multithreading (Memory Models, Atomics, CAS loops, non-blocking synchronization).
    - Basics of Compiler Construction (familiarity with LLVM IR is a significant plus).
    - Understanding of Virtual Memory and Shadow Memory concepts.
- **Tools:** C/C++, LLVM, Linux.

6. Expected Outcome

- A fork of the LLVM/TSan RTL repository with the implemented feature.
- A demonstration where TSan reports: `WARNING: ThreadSanitizer: ABA detection (potential logic error on atomic ptr 0x...).`
- A technical report analyzing the performance overhead of this solution.